

Understanding the Expressivity of Multi-Headed Attention

Mentor: Karthik Viswanathan, vkarthik095@gmail.com

Overview

This project studies the expressivity of multi-headed attention, a core primitive in modern transformers. We begin with a concrete question: how many parallel attention heads H are required to compute a given Boolean function

$$f : \{0, 1\}^n \rightarrow \{0, 1\}?$$

As a proof of concept, we have shown that n -bit parity requires exactly n heads, while AND and OR require only one. The goal of this project is to extend this analysis from these basic examples to broader classes of Boolean functions, and eventually to a more general theory of attention-head expressivity.

A reader-friendly introduction to the flavor of the project is available on LessWrong: *How many attention heads do you need to do XOR?*

Background and Significance

Multi-headed attention is central to transformer architectures, yet its expressive power remains poorly understood. By treating the number of heads H as a complexity measure for Boolean functions, we can understand more rigorously the expressivity of multihead attention. This perspective creates a bridge between Boolean complexity theory and transformer interpretability. It connects naturally to classical notions such as threshold degree, rational degree, and circuit complexity, while also relating to recent work on parity in transformers, including Kozachinskiy et al., 2026. More broadly, the project aims to develop mathematical tools and a Lean codebase for understanding what transformer components can and cannot represent.

Approach: A Mech-Interp Dojo

Recent articles such as Galois’s *Claude Can Sometimes Prove It* suggest that LLM coding agents can now discharge many non-trivial Lean proofs. Lean’s verification layer provides a safeguard against the natural-language hallucinations common in LLM-generated proofs, making it a promising environment for agent-assisted formalization.

The idea of this project is to formalize the relevant attention-head expressivity questions in Lean and use coding agents to handle much of the proof engineering, while humans focus on theorem selection, abstraction design, and research direction. A concrete deliverable is a *mech-interp dojo*: a Lean library and prompting scaffold that allows LLM agents to productively prove theorems in this project.

The mentor’s role is to provide research taste, mathematical framing, and mechanistic-interpretability context. The mentee’s Lean-side work will focus on designing the dojo: choosing useful abstractions, structuring the codebase, and setting up proofs so that downstream results can chain together effectively. These are precisely the parts that current agents do not yet handle well on their own.

Prerequisites

- Familiarity with Lean.
- Familiarity with coding agents such as Claude Code, Codex, or similar tools.
- Basic familiarity with mechanistic interpretability is useful but not required.